

Módulo 3 · Conocimientos en BBDD

Estrategias de optimización SQL y explicación de las decisiones técnicas.

1. Consulta lenta a optimizar

Contexto: tabla `orders` con **10.000.000** de filas y `customers` con **500.000**, consultando las órdenes de clientes de **México**:

```
SELECT o.order_id, o.order_date, c.customer_name, o.total_amount
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
WHERE c.country = 'México';
```

Sin índices, el motor hace **Seq Scan** sobre 10M de órdenes y 500K de clientes, con un JOIN costoso entre ambos. A continuación las estrategias aplicadas y su justificación.

2. Estrategia 1 — Índices compuestos y cubrientes (Index-Only Scan)

```
-- Filtrar solo clientes de México sin ir a la tabla (covering)
CREATE INDEX idx_customers_mexico ON customers (country, customer_id) INCLUDE (customer_name);

-- Cubrir el JOIN y las columnas proyectadas de orders
CREATE INDEX idx_orders_customer_covering ON orders (customer_id) INCLUDE (order_id, order_date,
total_amount);
```

Decisión y justificación: el índice `customers(country, customer_id)` convierte el filtro `country = 'México'` en un *Index Range Scan* y, al incluir `customer_name`, se vuelve **Index-Only Scan** (no toca páginas de datos). El índice cubriente sobre `orders.customer_id` permite resolver la proyección directamente desde el índice, de modo que el motor puede usar *Hash Join* o *Merge Join* sobre estructuras pequeñas: desaparece el Seq Scan de 10M de filas.

3. Estrategia 2 — Particionamiento + vista materializada

```
-- Partición por rango temporal de orders (Partition Pruning)
CREATE TABLE orders (...) PARTITION BY RANGE (order_date);

-- (alternativa) Partición por lista de customers.country

-- Vista materializada para el reporte recurrente de México
CREATE MATERIALIZED VIEW mv_mexico_orders AS
SELECT o.order_id, o.order_date, c.customer_name, o.total_amount
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
WHERE c.country = 'México';

CREATE UNIQUE INDEX idx_mv_mexico_orders ON mv_mexico_orders (order_id);

-- Refresco concurrente (sin bloquear lecturas)
REFRESH MATERIALIZED VIEW CONCURRENTLY mv_mexico_orders;
```

Decisión y justificación: el particionamiento (por `order_date` o `country`) permite *Partition Pruning* — el optimizador descarta particiones completas según el filtro. La **vista materializada** precalcula el resultado exacto del reporte de México; las consultas recurrentes leen milisegundos desde ese snapshot y el refresco `CONCURRENTLY` se hace sin bloquear a los usuarios. Se usa cuando el reporte es frecuente y tolera una ligera desactualización.

4. Estrategia 3 — Reestructuración CTE con filtrado temprano + EXPLAIN ANALYZE

```
WITH mexico_customers AS (  
  SELECT customer_id, customer_name  
  FROM customers  
  WHERE country = 'México'  
)  
SELECT o.order_id, o.order_date, mc.customer_name, o.total_amount  
FROM mexico_customers mc  
JOIN orders o ON mc.customer_id = o.customer_id;  
  
-- Validación del plan real:  
EXPLAIN (ANALYZE, BUFFERS)  
WITH mexico_customers AS (...) SELECT ... ;
```

Decisión y justificación: reducir el conjunto de trabajo *antes* del JOIN: el CTE materializa primero los pocos clientes mexicanos y luego los junta con `orders`, minimizando filas exploradas en el lado grande. `EXPLAIN ANALYZE` permite confirmar el *plan real* (no estimado), ver el número de filas (rows), los buffers leídos y el tiempo por nodo, y así decidir datos de forma objetiva.

5. Decisión aplicada a la propia plataforma (DDL con constraints e índices)

La misma disciplina se aplicó al modelo de pólizas: constraints de integridad y regla de negocio en la base de datos (índice único condicional para "individual = 1 riesgo").

```
-- Regla de negocio en BBDD: póliza individual solo 1 riesgo activo  
CREATE UNIQUE INDEX idx_unico_riesgo_individual  
ON riesgos (poliza_id)  
WHERE (estado = 'ACTIVO')  
  AND poliza_id IN (SELECT id FROM polizas WHERE tipo = 'INDIVIDUAL');  
  
-- Consulta A: vencimientos en 30 días (filtro por fecha y estado)  
CREATE INDEX idx_polizas_vencimiento_estado  
ON polizas (fecha_fin, estado)  
INCLUDE (numero_poliza, canon_mensual);  
  
-- Consulta B: prima acumulada por tipo (solo ACTIVA)  
CREATE INDEX idx_polizas_tipo_estado  
ON polizas (tipo, estado)  
INCLUDE (canon_mensual, prima_total);  
  
-- JOIN frecuente pólizas ↔ riesgos  
CREATE INDEX idx_riesgos_poliza_id_estado ON riesgos (poliza_id, estado);
```

6. Resumen de decisiones técnicas

Estrategia	Cuándo usarla	Beneficio clave
Índices cubrientes	Consultas sobre tablas enormes con filtro selectivo y columnas conocidas	Index-Only Scan: elimina Seq Scan y accesos a datos
Particionado + MV	Reportes recurrentes y datos históricos que crecen	Partition Pruning y lecturas en ms sin bloquear
CTE + filtrado temprano	JOINS con una tabla mucho más grande que otra	Reducción del conjunto de trabajo antes del JOIN
EXPLAIN ANALYZE	Siempre antes de decidir	Evidencia empírica del plan y de los costos

Fuente: *spring-boot-backend/README.md* (Módulo 3) y pestaña BBDD & Git (*src/components/DatabaseAndGitView.tsx*) de la plataforma.